WRENFORD BOROUGH COUNCIL · STREET CARE OFFICE

# StreetCare AI

Every flow in the system, one simple architecture diagram each.

**What the system does, in one line:** it takes street defect reports (potholes, broken paving, blocked drains, overhanging trees, faulty street lights), checks each one against the council's rules, adds machine-learning estimates, lets AI agents write a recommendation, and then waits for a human Highways Inspector to decide.

## Flows in this document


1. Overall architecture
2. Setup and run order
3. Where the data comes from
4. Data cleaning
5. Model training
6. Knowledge base
7. Report screening
8. Reading attached documents
9. Predictions
10. Review workflow
11. AI agents
12. Human decision
13. What each decision produces
14. Sign-in and access control
15. API request
16. Chat assistant
17. Dashboard
18. Storage
19. End to end


## Colour key

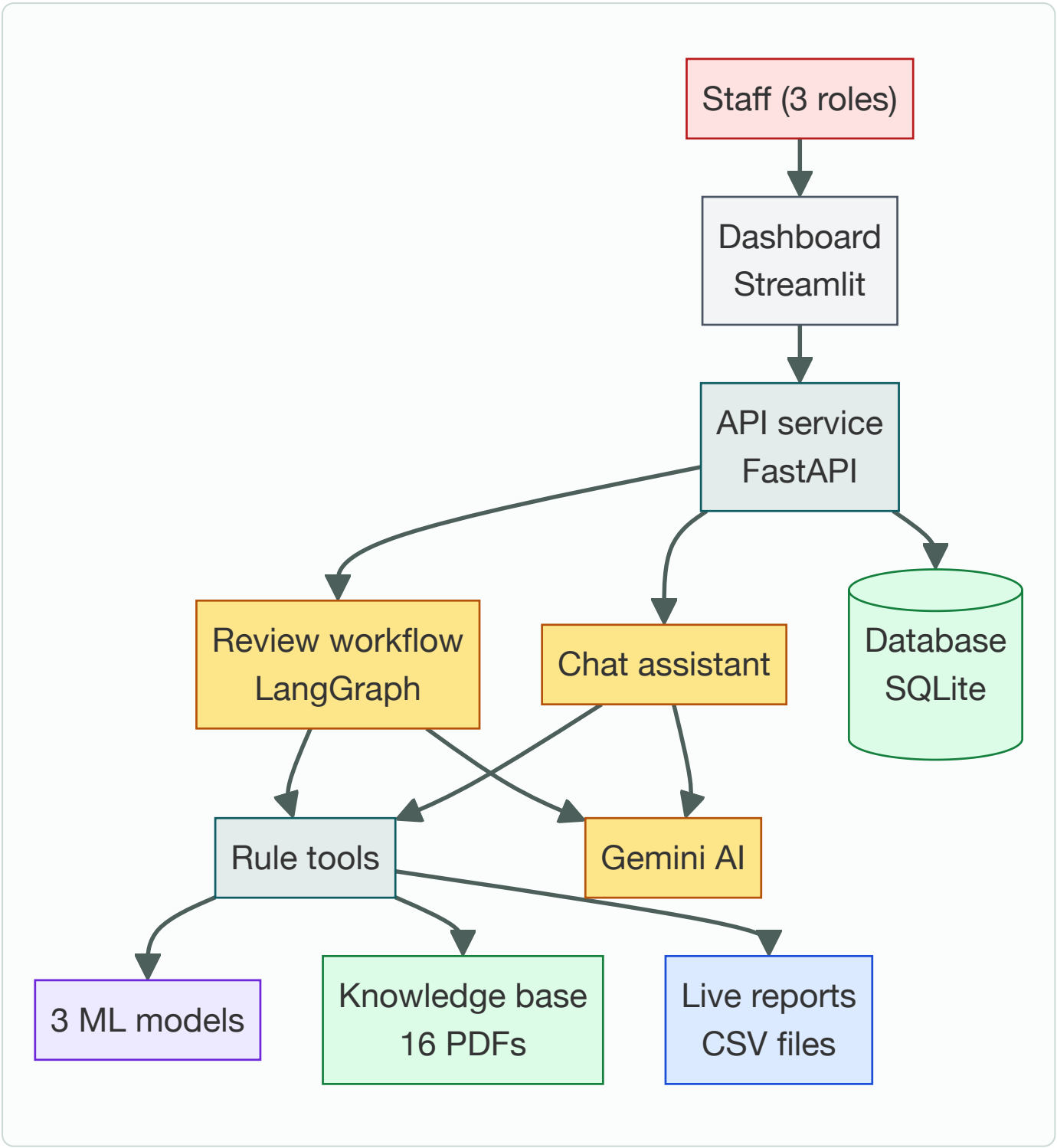
Input data · Code step · ML model · AI agent (Gemini) · Person / decision · Output / storage · Screen

> This document is based on the project specification (`StreetCare_AI.pdf`), `config.py` and the data files. The code modules are not written yet, so it shows the system as designed.

# 1 Overall architecture

**INPUT**
CSV data files, 16 council PDFs, settings

**OUTPUT**
A web dashboard where staff review and decide reports

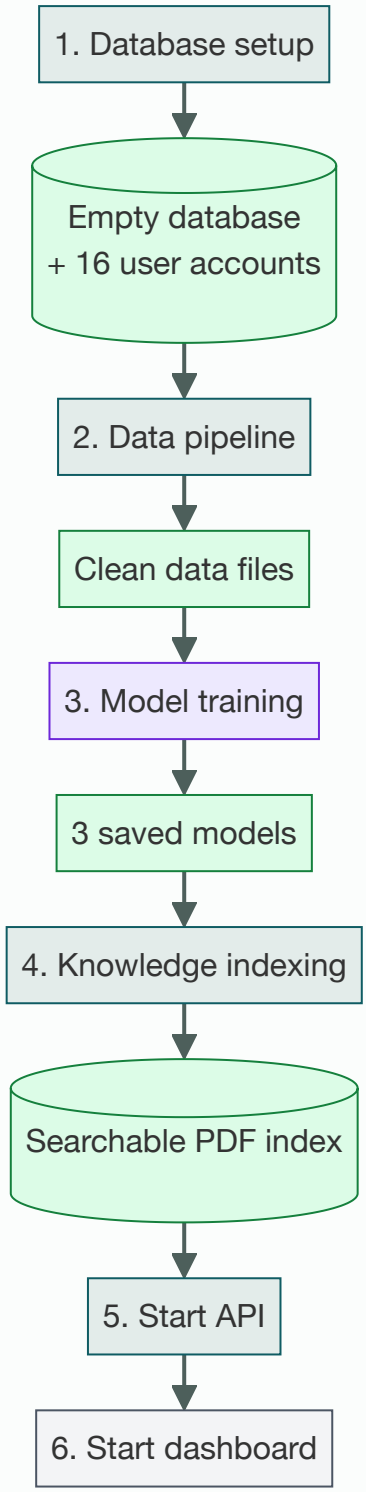



- Staff use the **dashboard**. The dashboard only talks to the **API**.
- The API runs the **review workflow** and the **chat**. Both use rule tools for facts and Gemini for writing.
- Everything that happens is saved in the **database**.

## 2 Setup and run order

**INPUT**
The project folder with data and settings

**OUTPUT**
A running API (port 8081) and dashboard (port 8080)

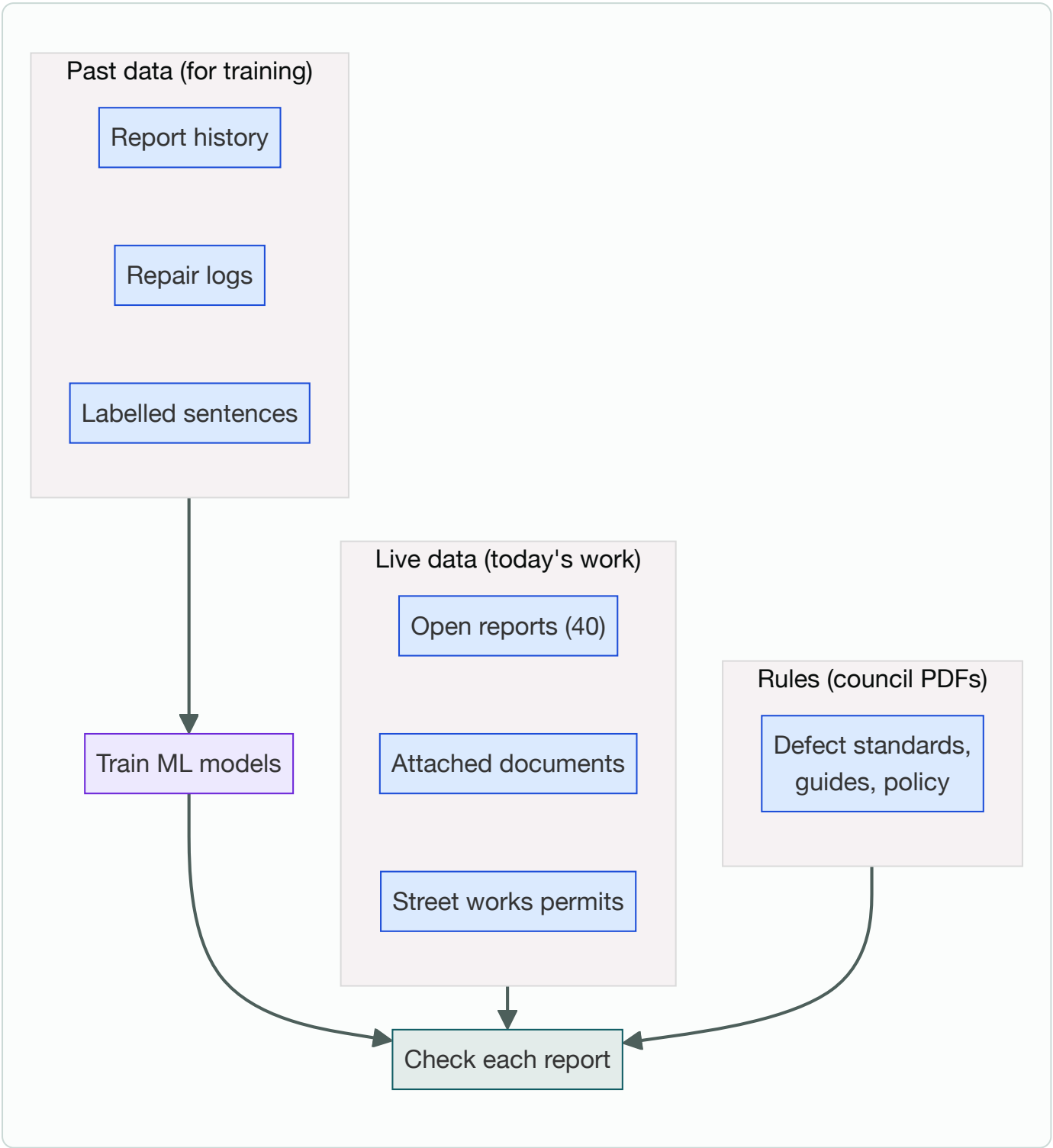



- Steps 1–4 are run **once**. They prepare everything the live system needs.
- Steps 5–6 start the live system. The API will not start if a model is missing.

# 3 Where the data comes from

**INPUT**
11 files in three groups

**OUTPUT**
Training data, the live work queue, and the rules

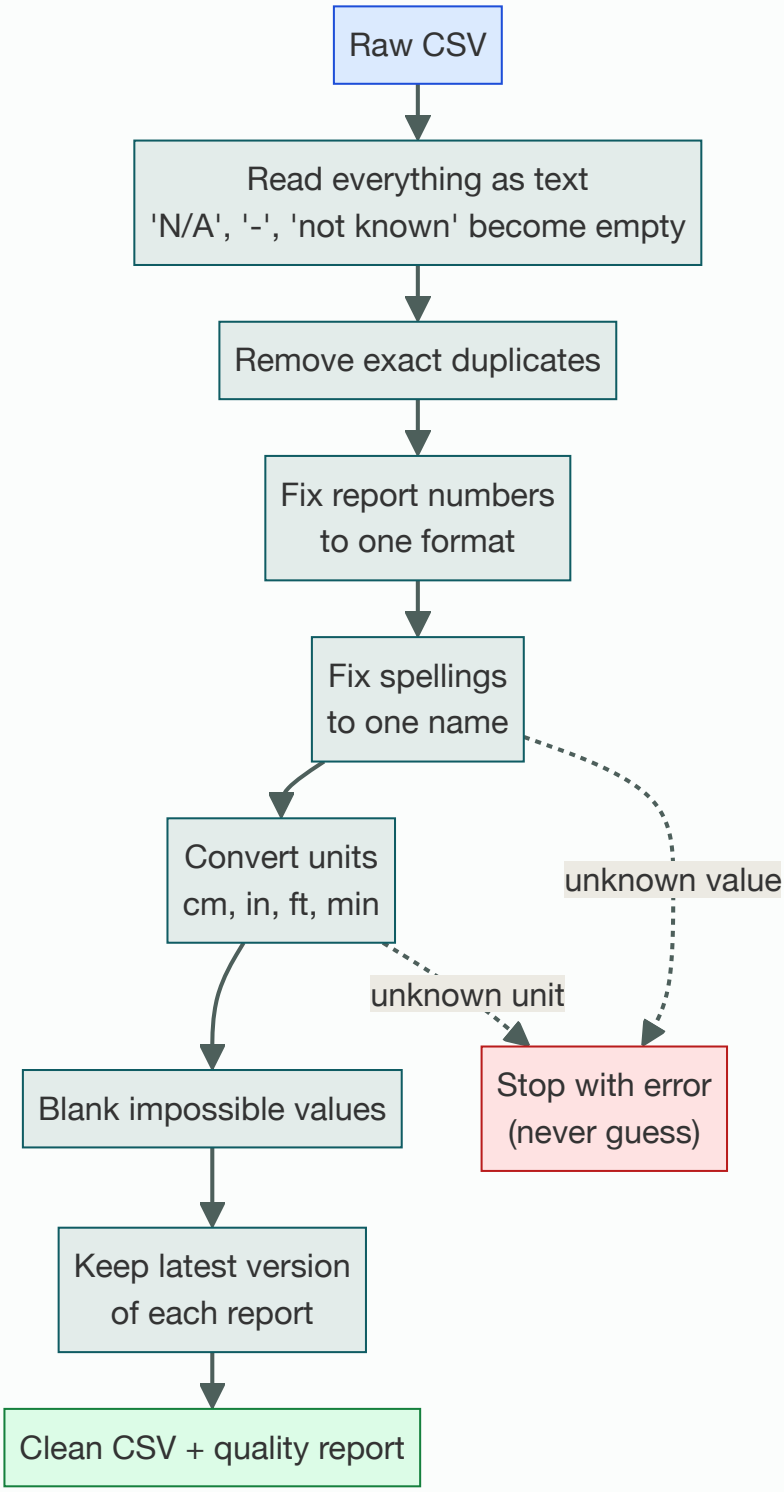



- **Past data** teaches the models.
- **Live data** is the queue of reports waiting for review.
- **PDFs** are the only place rules and limits come from. No rule is typed into the code.

# 4 Data cleaning

**INPUT**
Messy CSVs: "SC-50118", "4.5 cm", "210 min", "N/A", "CW"

**OUTPUT**
Clean CSVs: "SC0050118", "45 mm", "3.5 h", empty, "Carriageway"

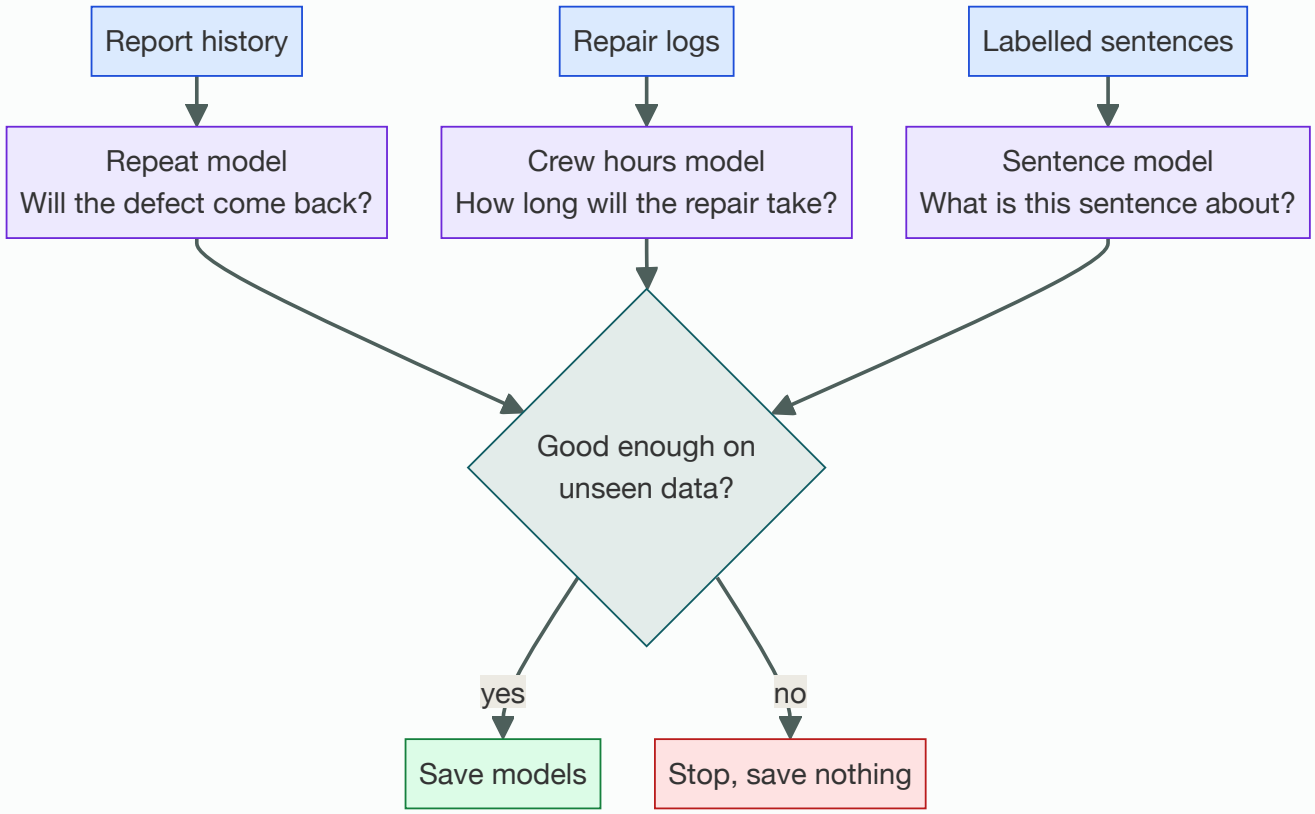



- Example: **32,738** history rows go in and **30,569** clean rows come out.
- A quality report records what was wrong and what was fixed.

# 5 Model training

**INPUT**
Clean history, repair logs and sentences

**OUTPUT**
3 saved models + their scores

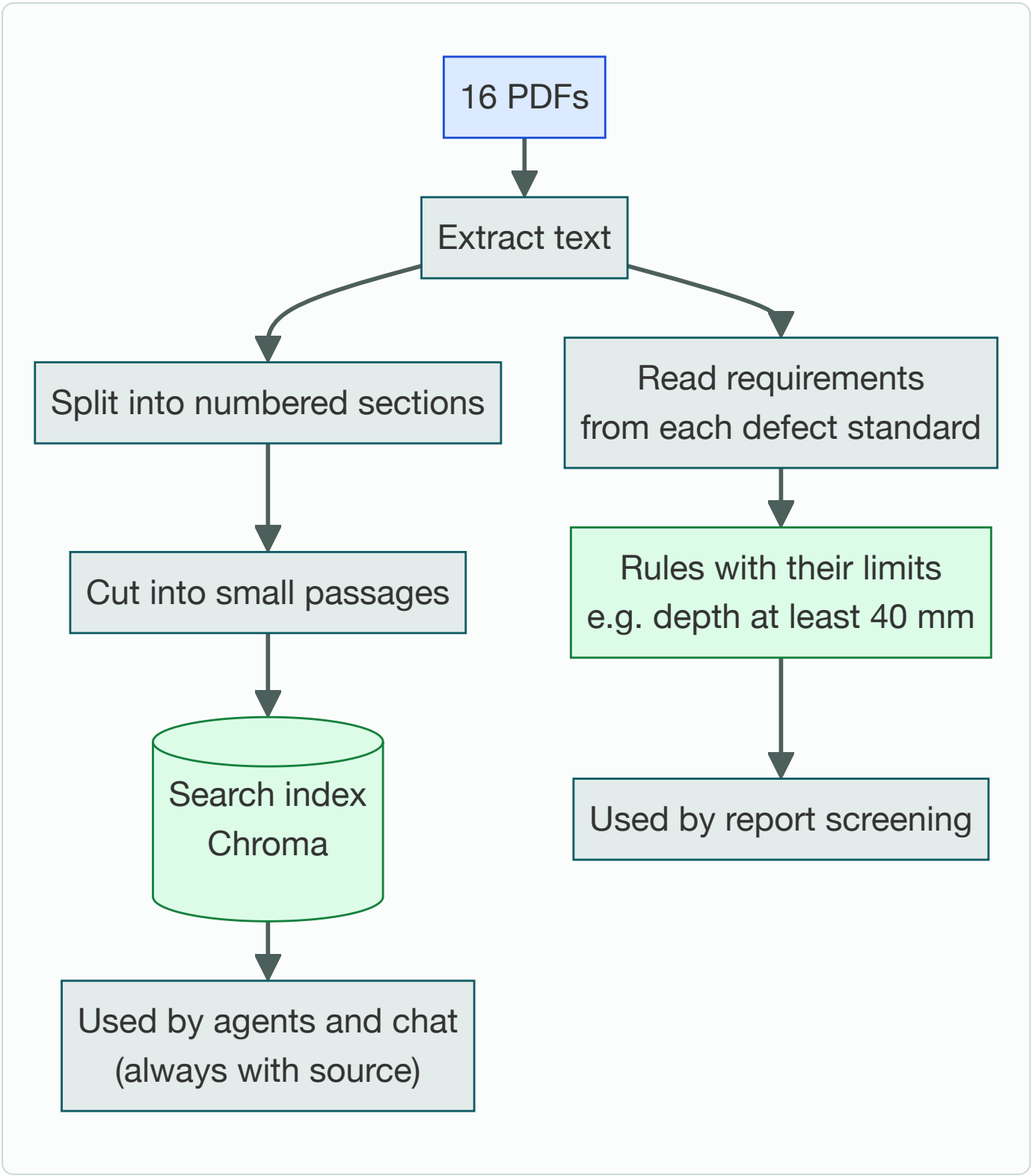



- Each model is tested on a separate set of data it never trained on.
- Minimum scores: repeat 0.80 ROC AUC, crew hours 0.86 $R^2$, sentence 0.95 F1.

## 6 Knowledge base

**INPUT**
16 council PDFs

**OUTPUT**
A searchable index + the rules of each defect type

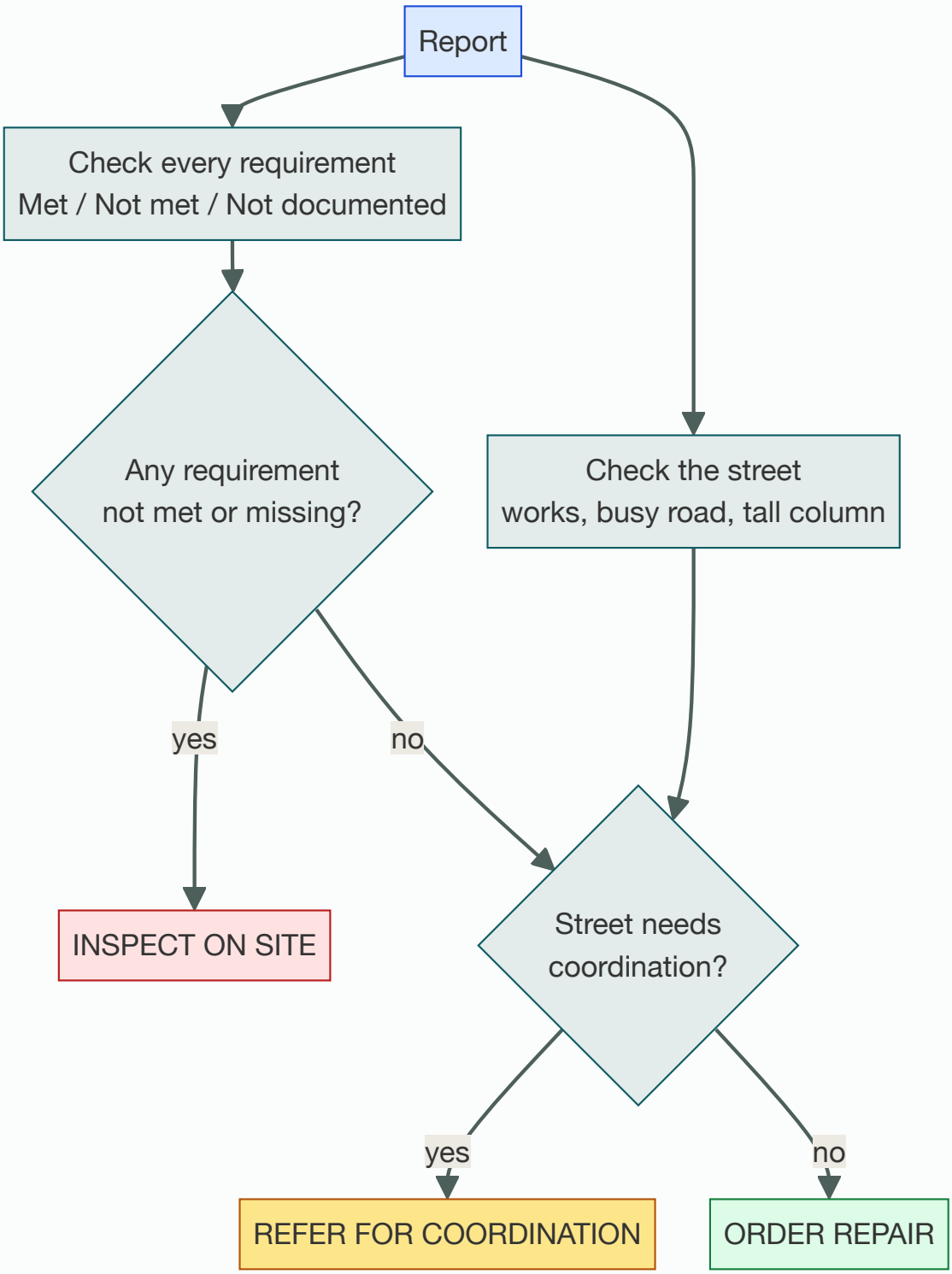



- Every passage remembers which document and section it came from.
- Each defect type's limits are read from **its own** standard.

# 7 Report screening

**INPUT**
One clean report + its defect standard + street works permits

**OUTPUT**
A suggested assessment: ORDER, INSPECT or REFER

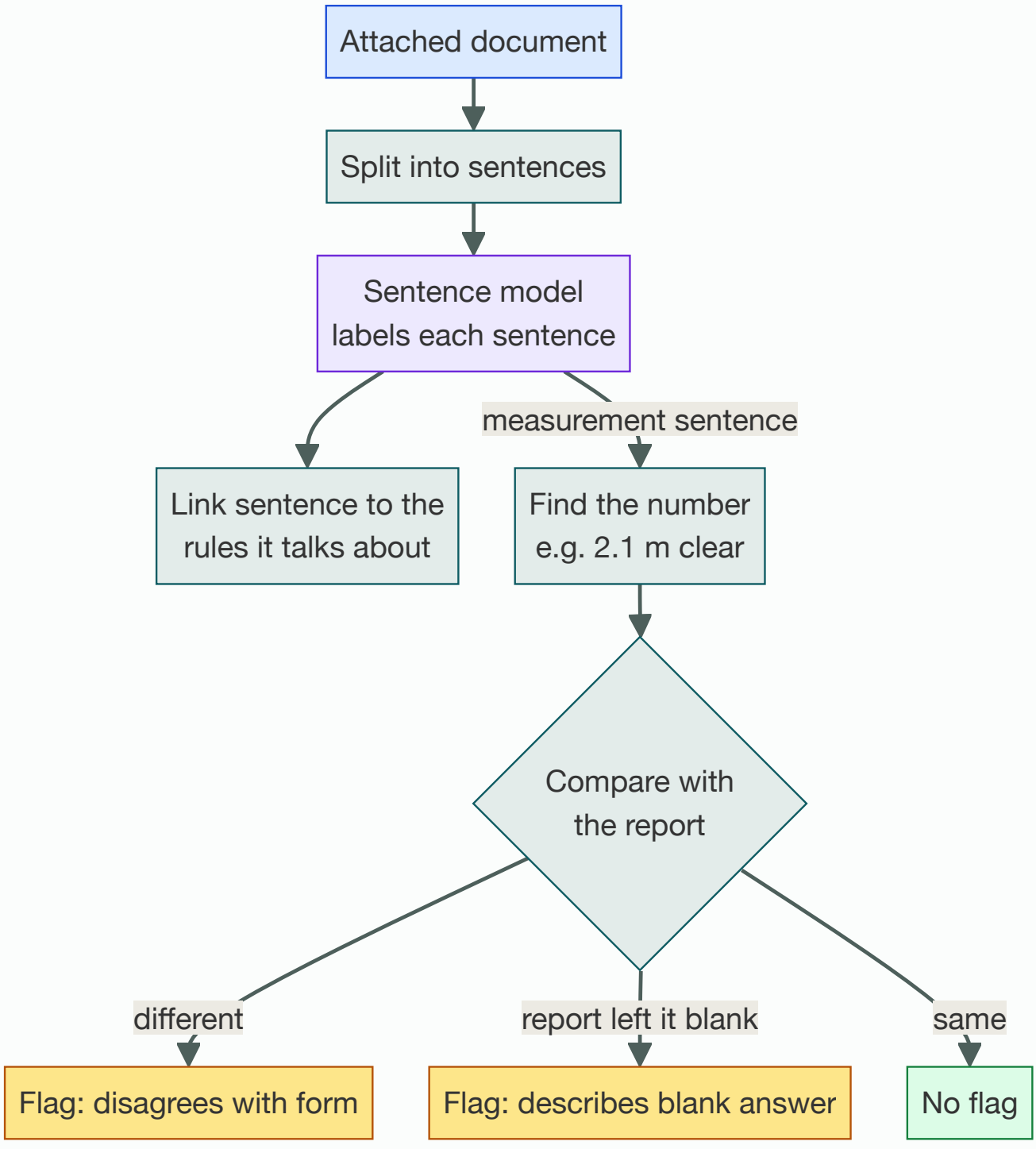



- Requirements are checked first, then the street.
- A missing answer never lets a report skip a check.

# 8 Reading attached documents

**INPUT**
Notes and descriptions attached to a report

**OUTPUT**
Each sentence labelled, plus warning flags

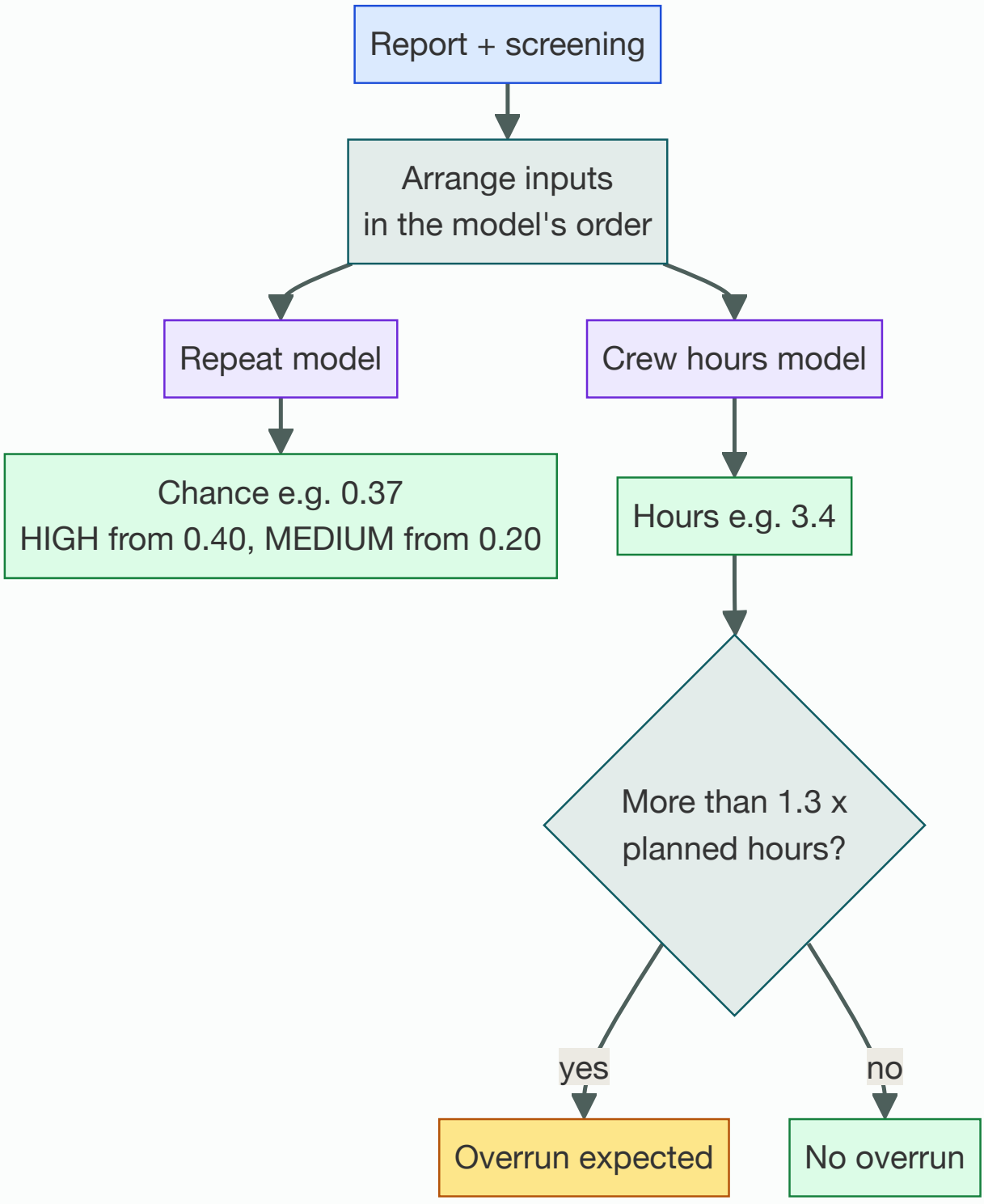



- Flags alert the inspector. They never change the report's answers.

# 9 Predictions

**INPUT**
Report answers + screening results

**OUTPUT**
Repeat chance (LOW / MEDIUM / HIGH) and expected crew hours

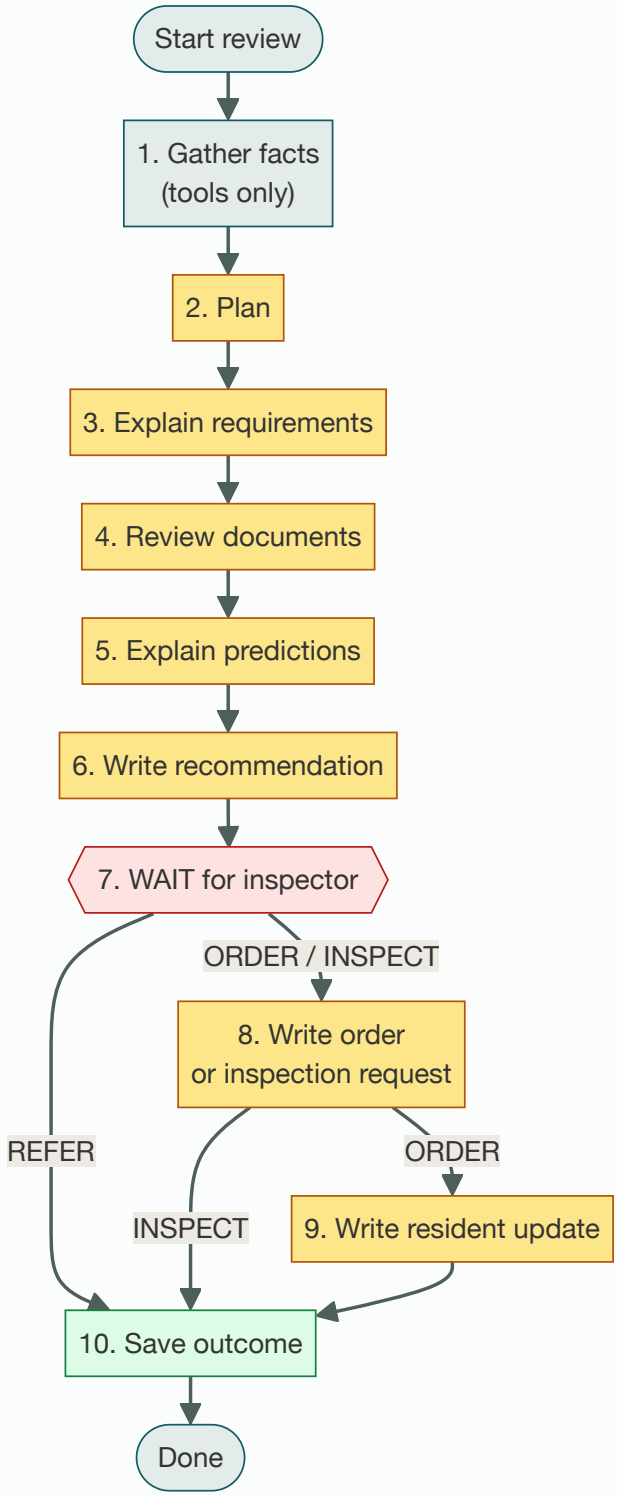



- Predictions only **inform** the inspector. They decide nothing.
- Predictions never appear in letters to residents or works orders.

# 10 Review workflow

**INPUT**
A report number + the coordinator's request

**OUTPUT**
A stored review, a decision, and the documents it calls for

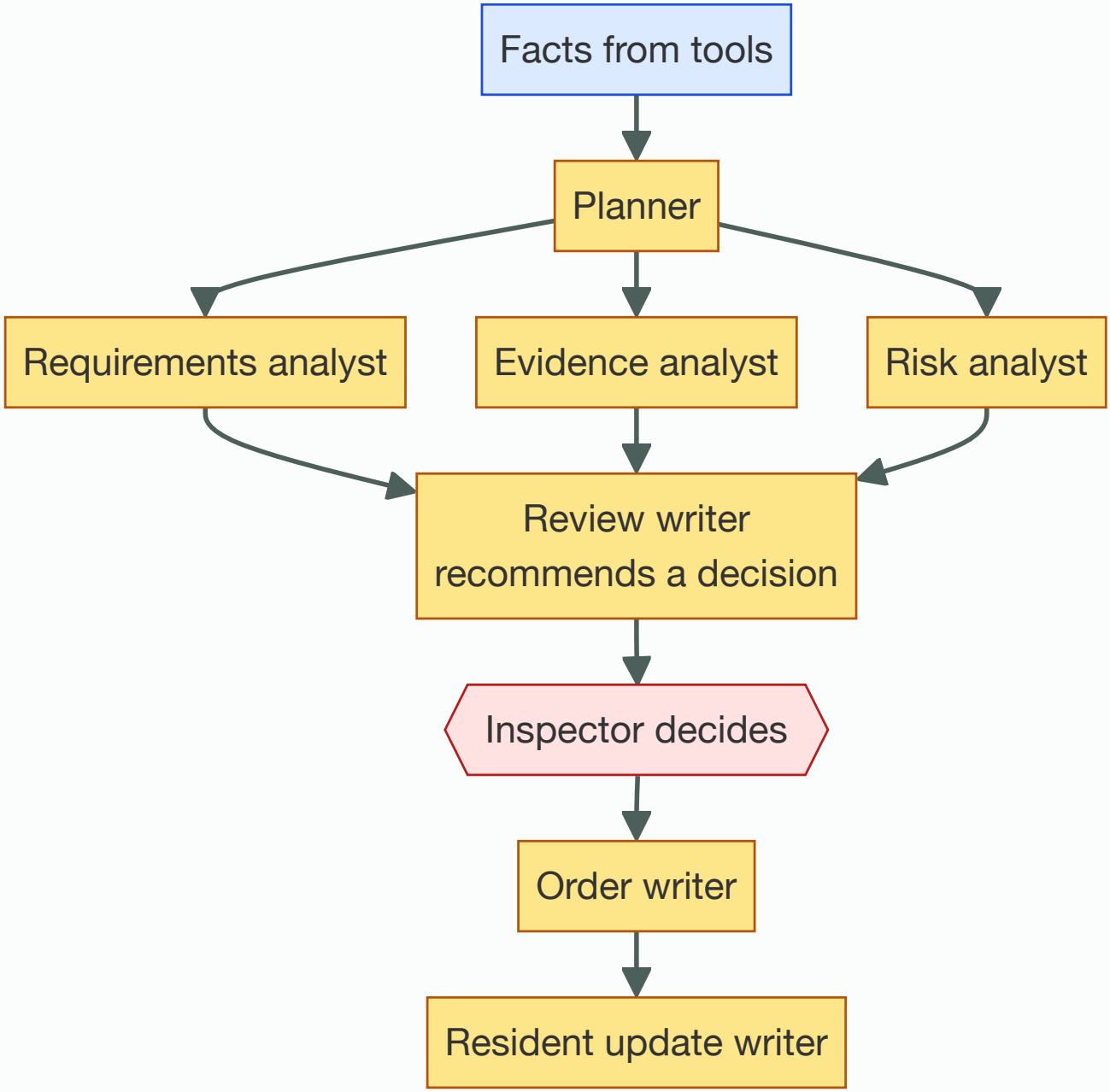



- Each yellow step is **one** Gemini call (5 before the wait, up to 2 after).
- The wait is saved to disk, so it survives a restart.

# 11 AI agents

**INPUT**
Facts from the tools + passages from the PDFs

**OUTPUT**
Written explanations and a recommended decision

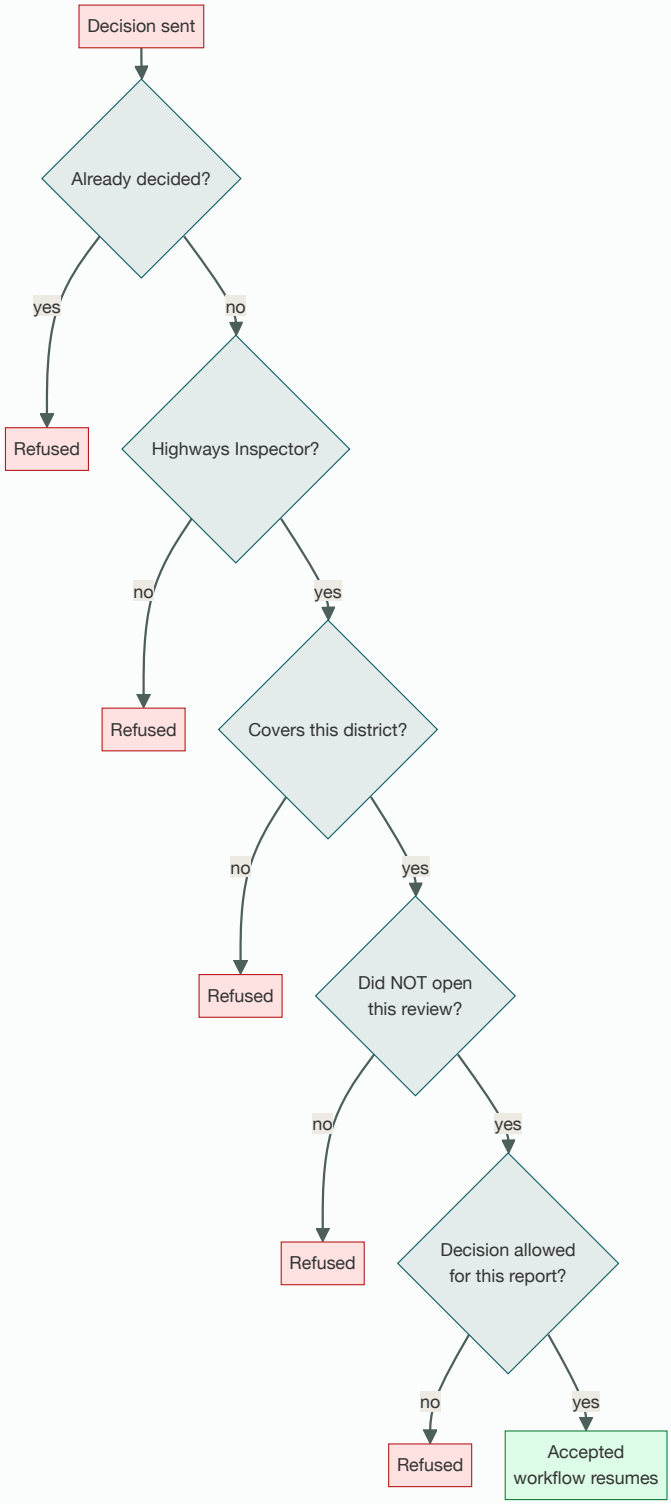



- Agents **explain**; they never calculate. Every number comes from the tools.
- Agents pass their work to each other through the shared workflow state.

# 12 Human decision

**INPUT**
An inspector's decision and reason

**OUTPUT**
Accepted (workflow continues) or refused with a reason

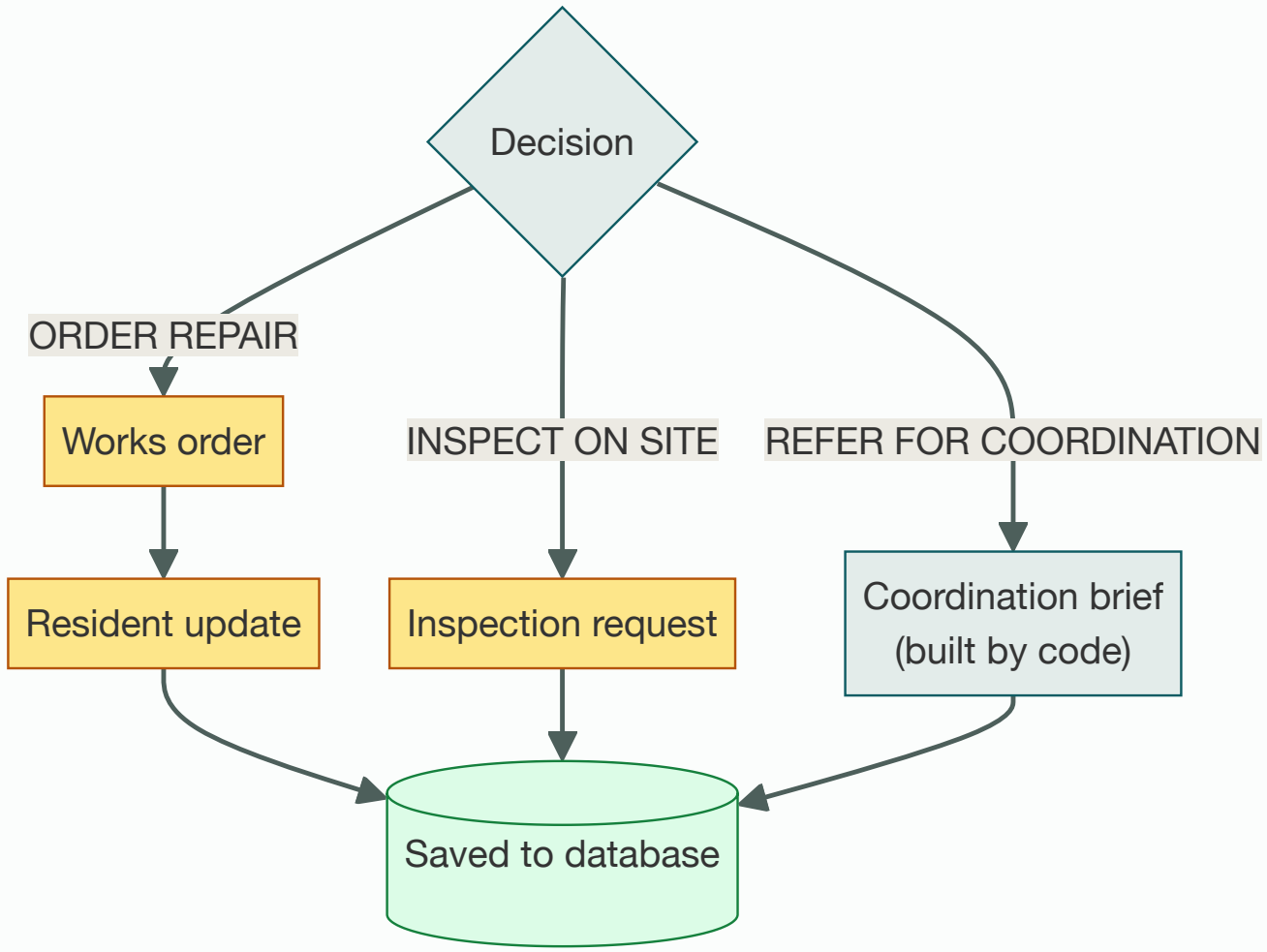



- Two inspectors cover each district, so one can check what the other opened.

## 13 What each decision produces

**INPUT**
The accepted decision

**OUTPUT**
The right documents, saved with the review

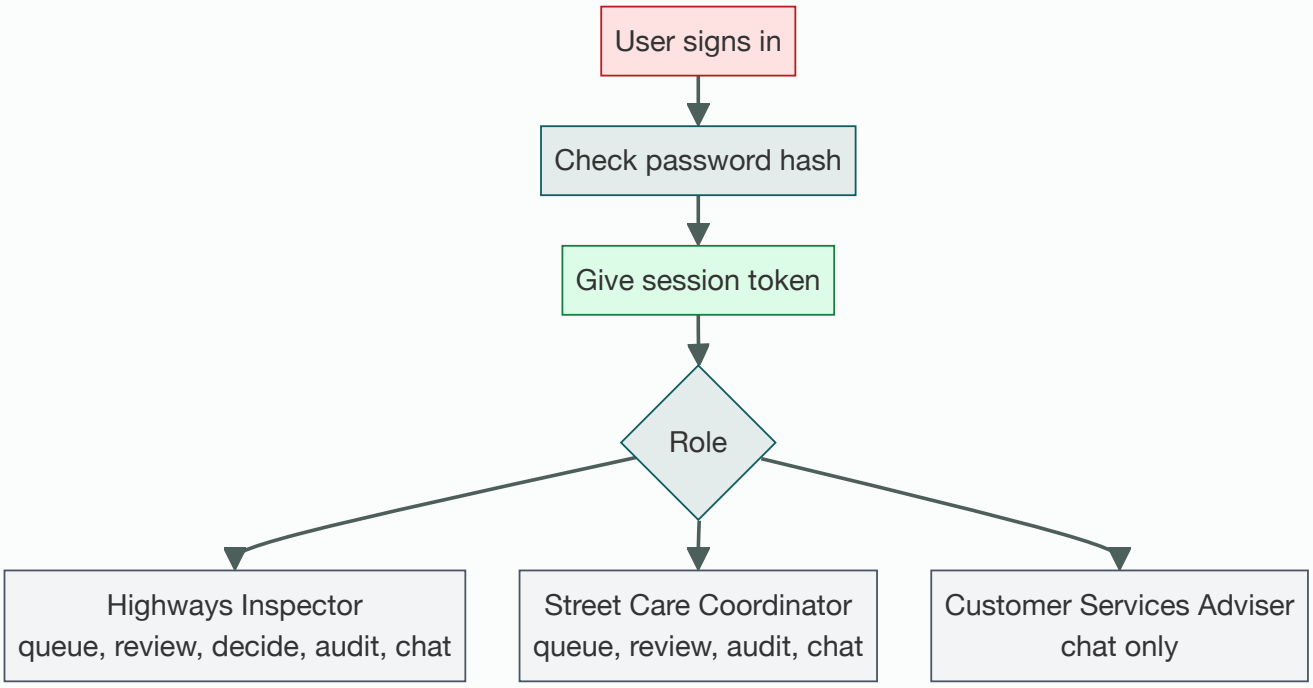



- ORDER uses 7 AI calls in total, INSPECT 6, REFER 5.
- Every document is signed "Street Care Office, Wrenford Borough Council".

# 14 Sign-in and access control

**INPUT**
Username + password, then a session token on every request

**OUTPUT**
Only the pages and actions the role allows

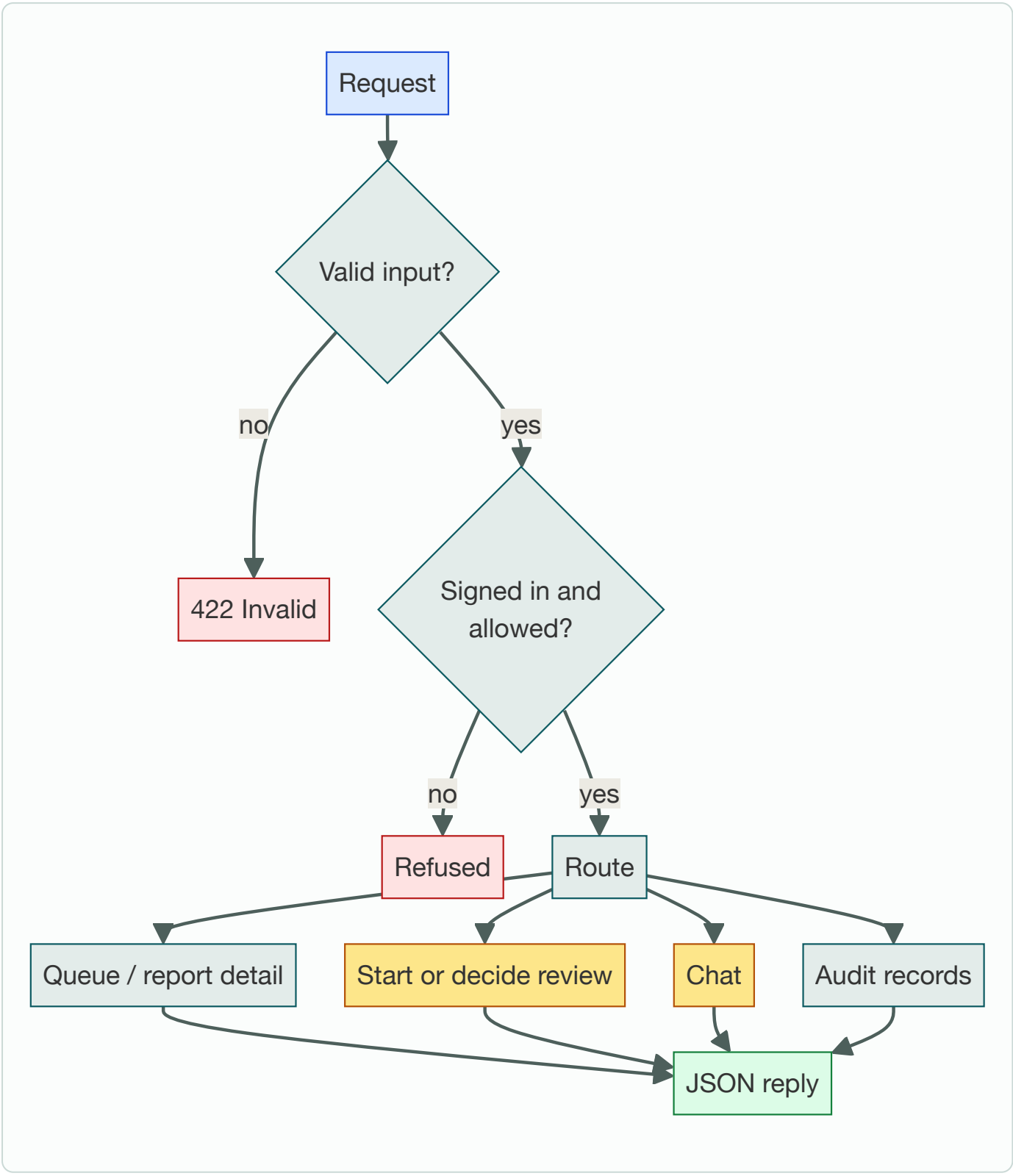



- The API itself refuses any request the role does not allow, not just the dashboard.

# 15 API request

**INPUT**
An HTTP request from the dashboard

**OUTPUT**
A JSON reply, or an error code

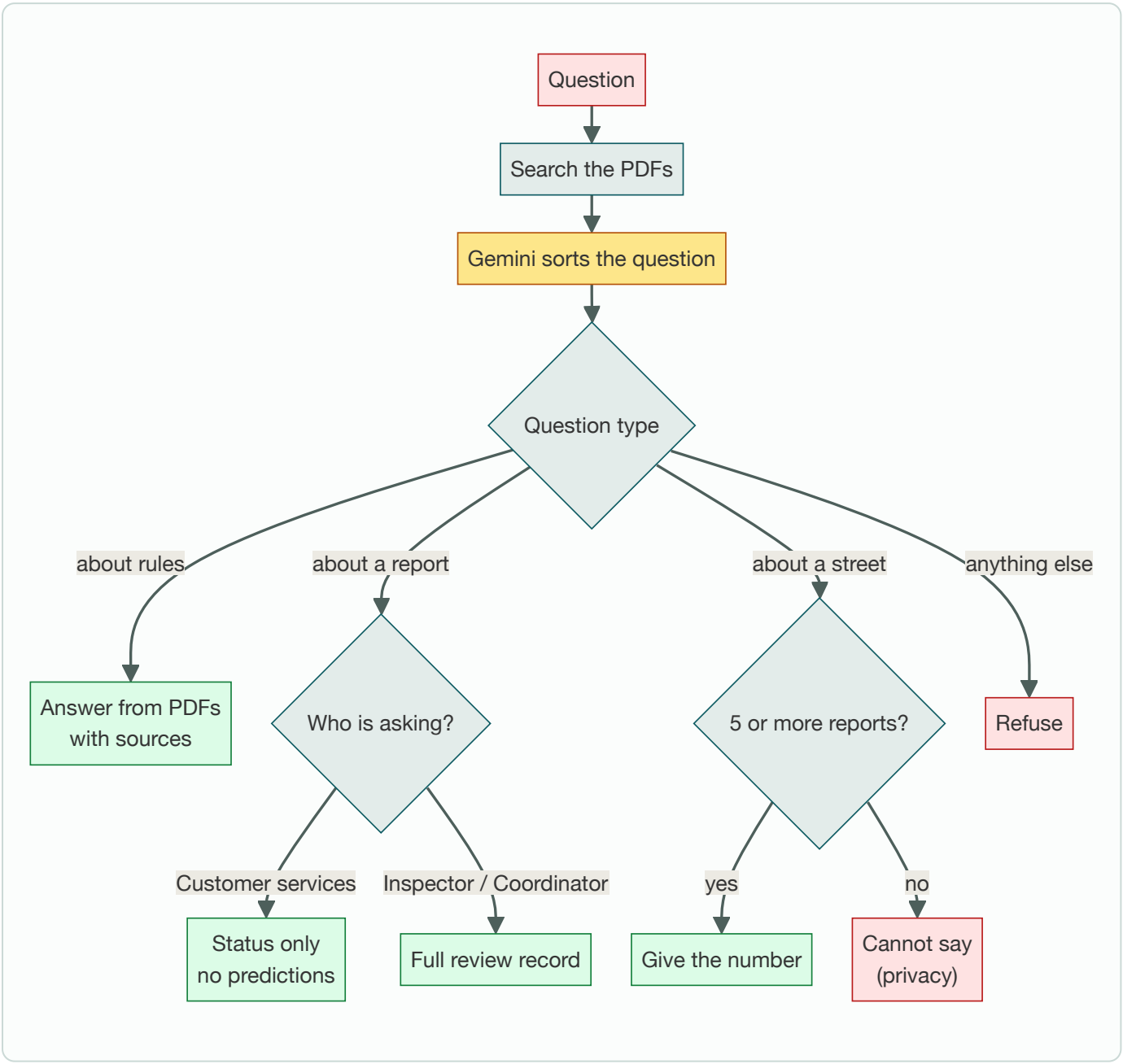



- Only starting a review, deciding a review and asking the chat can use AI calls.

# 16 Chat assistant

**INPUT**
A question typed by any signed-in user

**OUTPUT**
An answer with its sources, or a polite refusal

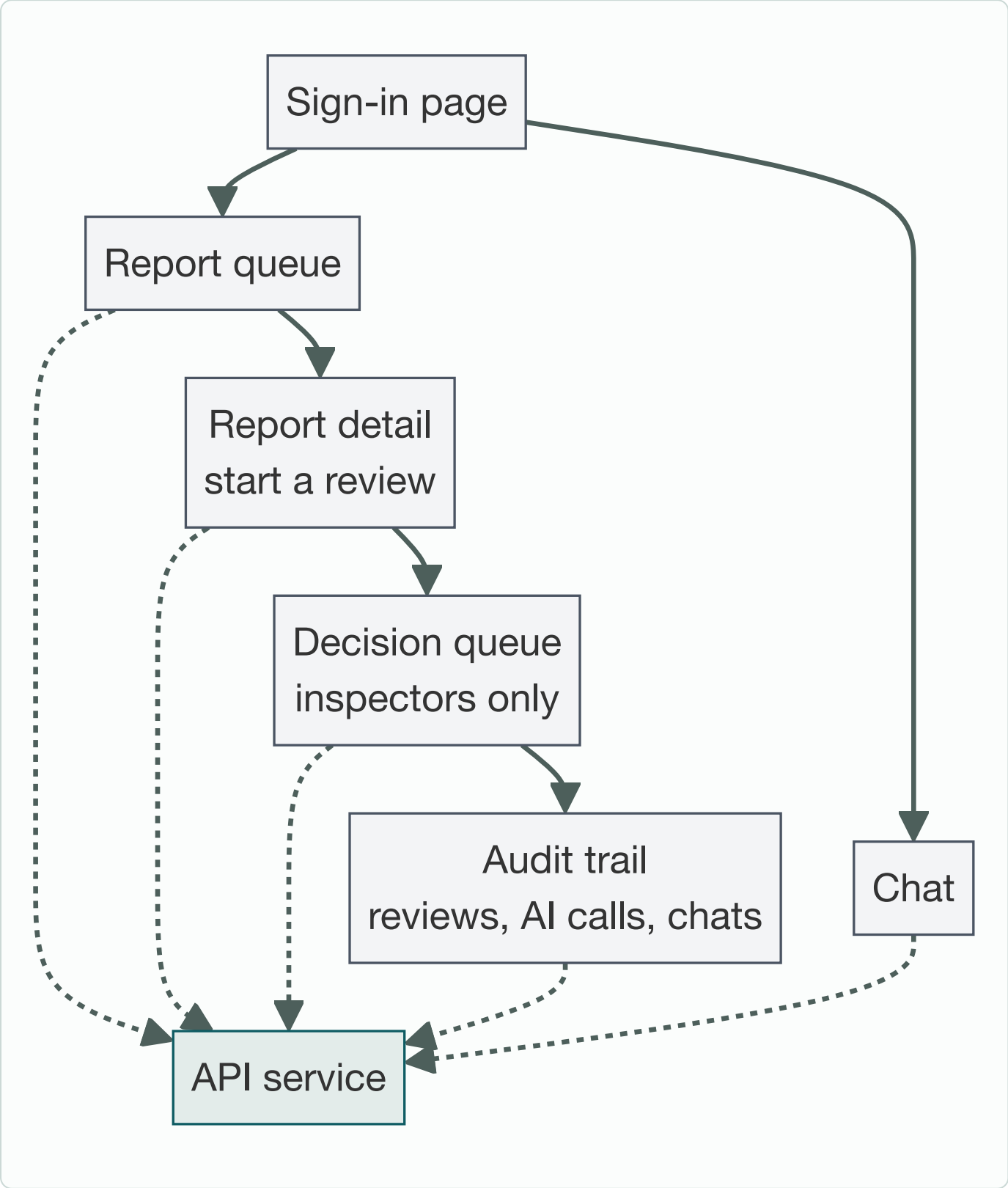



- Answers about reports and streets are written by code, not by the AI.
- Every question and answer is saved for audit.

## 17 Dashboard

**INPUT**
A signed-in user

**OUTPUT**
The pages for that user's role

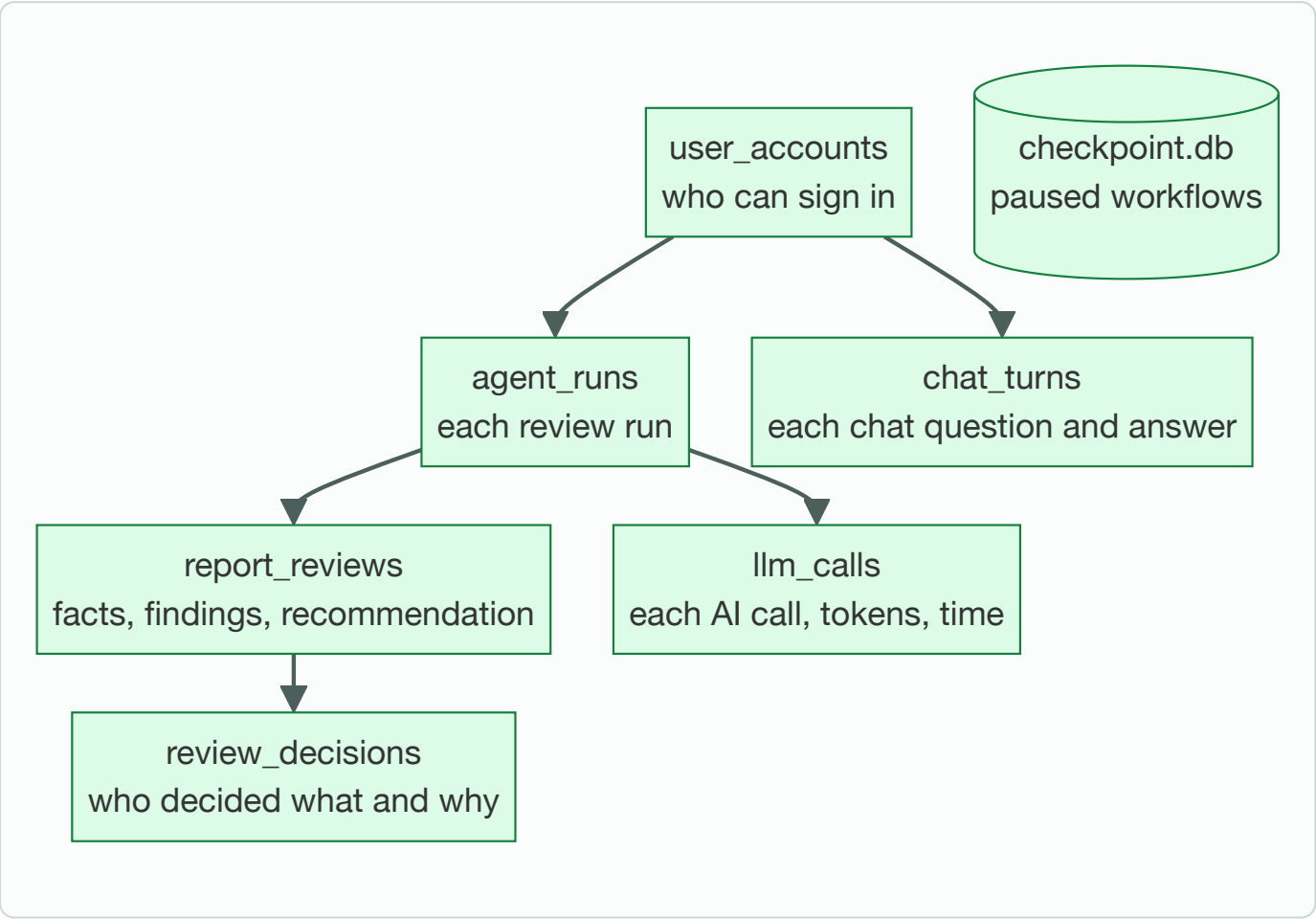



- The dashboard shows data; it holds no rules. Every number comes from the API.

## 18 Storage

**INPUT**
Everything the system does

**OUTPUT**
A full audit trail in 6 tables

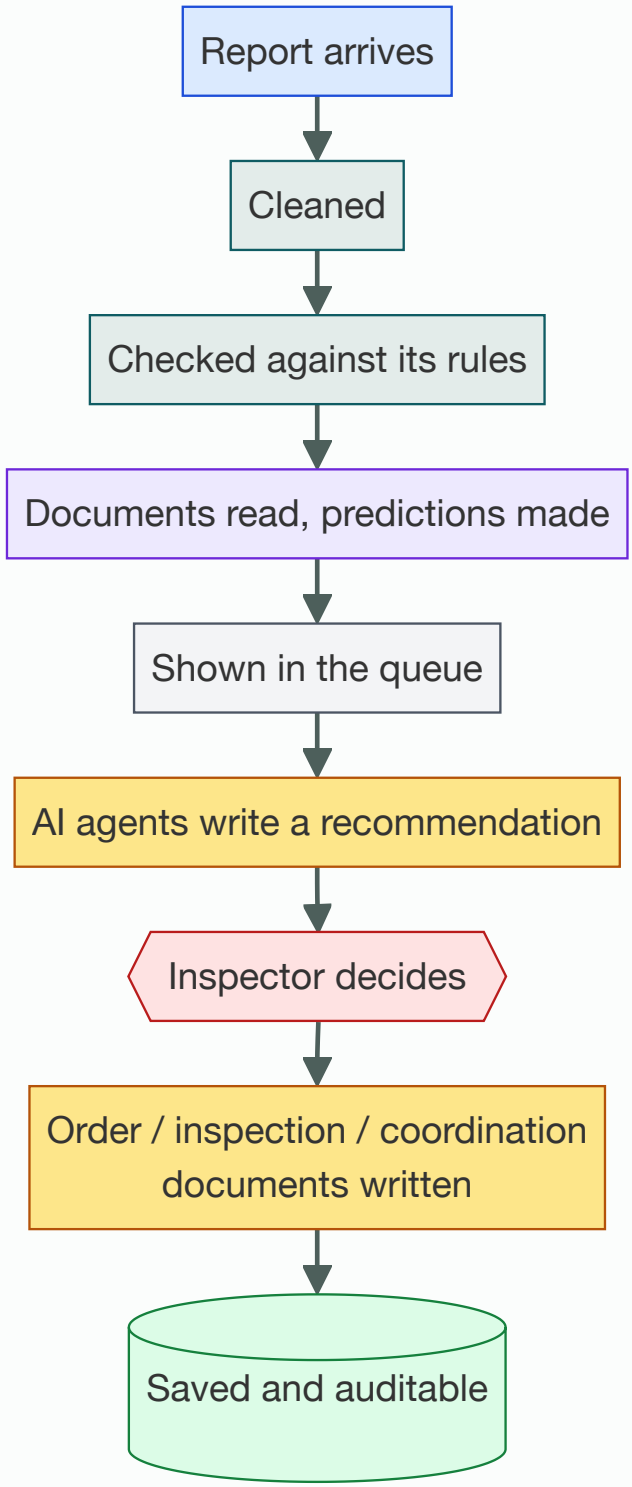



- Main database: `database/streetcare.db`. Paused workflows: `database/checkpoint.db`.

## 19 End to end

**INPUT**
A messy report from the website, app or phone

**OUTPUT**
A decided report with its documents, fully recorded




- **Rules come from PDFs. Tools calculate. AI explains. A person decides. Everything is recorded.**